

Tarea 1

Integrantes:

- Sofía Salinas Rico - ssalinas@unal.edu.co
- Camilo Chitivo Cerinza - cchitivo@unal.edu.co

Ejercicios

Considere $\Omega = [-2, 2] \times [-2, 2]$ y $C1 = \{(x, y) \in \Omega : x^2 + y^2 \leq 1\}$

1. Aproximar la probabilidad $P(C1)$.

Recuerde que $P(C1) = \frac{\pi}{16} \approx 0.19634954$

- Genere n puntos aleatorios uniformemente distribuidos en el cuadrado $[-2, 2] \times [-2, 2]$.
- Para cada punto generado (x_i, y_i) , determine si pertenece al conjunto $C1$.
- Utilice la frecuencia relativa de los puntos que pertenecen al círculo para obtener una aproximación $\hat{P}_n(C1)$ de $P(C1)$.
- Calcule el error absoluto $|P(C1) - \hat{P}_n(C1)|$.
- Realice la simulación para los siguientes tamaños de muestra:

$$n = 10^2, 10^3, 10^4, 10^5, 10^6.$$

- Presente los resultados en una tabla que contenga, para cada valor de n :

n , número de puntos dentro del círculo, $\hat{P}_n(C1)$, Error absoluto.

```
In [1]: import numpy as np
from math import pi
from tabulate import tabulate
```

```
In [2]: def get_approx(n: int):

    points = np.random.uniform(-2, 2, size=(n, 2)) # size inidica la forma del arreglo, n filas y 2 columnas

    x = points[:, 0] # Toma todos los valores de la primera columna
    y = points[:, 1] # Toma todos los valores de la segunda columna

    result = x**2 + y**2 <= 1 # Forma un arreglo con valores booleanos cuando se cumple o no la condición

    inside = np.sum(result)

    estimated = np.mean(result)

    return inside, estimated
```

```
In [3]: P_C1 = pi/16

data = []

for i in range(2, 7):
    n = 10**i
    inside, estimated = get_approx(n)
    error = round(abs(P_C1 - estimated), 8)
    data.append([f"10^{i}", inside, estimated, error])
```

```
In [4]: print(tabulate(
    data,
    headers=["n", "points inside", "P_n(C1) (E)", "error abs"],
    tablefmt="github"
))
```

n	points inside	P_n(C1) (E)	error abs
10 ²	22	0.22	0.0236505
10 ³	210	0.21	0.0136505
10 ⁴	1951	0.1951	0.00124954
10 ⁵	19541	0.19541	0.00093954
10 ⁶	197127	0.197127	0.00077746

2. Aproximar el número π

Utilice los resultados obtenidos en el punto anterior para construir una aproximación de π (3.14159265).

Un estimador $\hat{\pi}_n$ de π se obtiene de la siguiente forma:

$$\hat{\pi}_n := 16\hat{P}_n(C1)$$

¿Por qué se obtiene este estimador?

a) Calcule las aproximaciones de π para los valores de n dados en 1-e).

b) Presente los resultados en una tabla que contenga:

n , $\hat{P}_n(C1)$, $\hat{\pi}$, error absoluto.

El estimador se obtiene dado que $\frac{\pi}{16} \approx \hat{P}_n(C1)$, por lo cual podemos despejar y tener un estimador de π .

```
In [5]: data_2 = []

for row in data:
    n = row[0]
    p_n = row[2]
    pi_n = p_n*16
    error = round(abs(pi_n - pi), 8)
    data_2.append([n, p_n, pi_n, error])
```

```
In [6]: print(tabulate(
    data_2,
    headers=["n", "P_n(C1) (E)", "Pi_n (E)", "error"],
    tablefmt="github"
))
```

n	P_n(C1) (E)	Pi_n (E)	error
10^2	0.22	3.52	0.378407
10^3	0.21	3.36	0.218407
10^4	0.1951	3.1216	0.0199927
10^5	0.19541	3.12656	0.0150326
10^6	0.197127	3.15403	0.0124394

3. Considere los conjuntos

$$C_{\sqrt{2}} = \{(x, y) \in \Omega : x^2 + y^2 \leq 2\} \text{ y } C_2 = \{(x, y) \in \Omega : x^2 + y^2 \leq 2^2\}.$$

Usando la información y repitiendo los numerales anteriores, para cada conjunto C_i , $i = \sqrt{2}, 2$,

a) Calcule la probabilidad $P(C_i)$.

b) Obtenga las correspondientes aproximaciones $\hat{P}_n(C_i)$ de $P(C_i)$ para los valores de n dados en 1-e).

c) Deduzca un estimador de π a partir de $\hat{P}_n(C_i)$.

d) Obtenga las correspondientes aproximaciones $\hat{\pi}_n$ y calcule los errores absolutos.

e) Construya una tabla que muestre, para cada n , las aproximaciones de π obtenidas con los tres radios:

$$n, \hat{\pi}_n(r=1), \hat{\pi}_n(r=\sqrt{2}), \hat{\pi}_n(r=2)$$

f) Compare los resultados obtenidos para los diferentes radios. ¿Cuál de los tres radios parece proporcionar mejores aproximaciones de π ? ¿Por qué puede suceder esto?

Teniendo la distribución uniforme, la probabilidad de $P(C_{\sqrt{2}}) = \frac{\pi\sqrt{2}^2}{16} = \frac{\pi}{8}$ y la probabilidad de $P(C_2) = \frac{\pi 2^2}{2^4} = \frac{\pi}{4}$

a) Probabilidades exactas.

Como (X, Y) es uniforme en $\Omega = [-2, 2] \times [-2, 2]$, que tiene área $4^2 = 16$, para un radio $r \leq 2$ el conjunto $C_r = \{(x, y) \in \Omega : x^2 + y^2 \leq r^2\}$ es un círculo completo de área πr^2 , luego

$$P(C_r) = \frac{\pi r^2}{16}.$$

En particular:

$$P(C_{\sqrt{2}}) = \frac{2\pi}{16} = \frac{\pi}{8} \approx 0.39269908, \quad P(C_2) = \frac{4\pi}{16} = \frac{\pi}{4} \approx 0.78539816.$$

b) Aproximaciones $\hat{P}_n(C_r)$.

Se generaliza la función del punto 1 para un radio r cualquiera: es el mismo procedimiento (generar n puntos uniformes en el cuadrado y contar cuántos cumplen la condición), sólo cambia la condición $x^2 + y^2 \leq r^2$.

```
In [7]: def get_approx_r(n: int, r: float):  
  
    points = np.random.uniform(-2, 2, size=(n, 2)) # size indica la forma del arreglo, n filas y 2 columnas  
  
    x = points[:, 0] # Toma todos los valores de la primera columna  
    y = points[:, 1] # Toma todos los valores de la segunda columna  
  
    result = x**2 + y**2 <= r**2 # Igual que antes, pero la condición ahora depende del radio r
```

```

    inside = np.sum(result)

    estimated = np.mean(result)

    return inside, estimated

```

```

In [8]: radios = [("sqrt(2)", np.sqrt(2)), ("2", 2)]

data_3 = {}

for label, r in radios:
    P_Ci = pi*r**2/16
    rows = []
    for i in range(2, 7):
        n = 10**i
        inside, estimated = get_approx_r(n, r)
        error = round(abs(P_Ci - estimated), 8)
        rows.append([f"10^{i}", inside, estimated, error])
    data_3[label] = rows

```

```

In [9]: for label, r in radios:
    print(f"r = {label}    P(C_r) = {pi*r**2/16:.8f}")
    print(tabulate(
        data_3[label],
        headers=["n", "points inside", "P_n(C_r) (E)", "error abs"],
        tablefmt="github"
    ))
    print()

```

r = sqrt(2) P(C_r) = 0.39269908			
n	points inside	P_n(C_r) (E)	error abs
10^2	38	0.38	0.0126991
10^3	381	0.381	0.0116991
10^4	3900	0.39	0.00269908
10^5	39230	0.3923	0.00039908
10^6	392409	0.392409	0.00029008

r = 2 P(C_r) = 0.78539816			
n	points inside	P_n(C_r) (E)	error abs
10^2	85	0.85	0.0646018
10^3	771	0.771	0.0143982
10^4	7812	0.7812	0.00419816
10^5	78542	0.78542	2.184e-05
10^6	785792	0.785792	0.00039384

c) Estimador de π .

De $P(C_r) = \frac{\pi r^2}{16}$ se despeja $\pi = \frac{16 P(C_r)}{r^2}$, por lo cual un estimador natural es

$$\hat{\pi}_n := \frac{16}{r^2} \hat{P}_n(C_r).$$

Para $r = 1$ esto da $16\hat{P}_n$ (el del punto 2), para $r = \sqrt{2}$ da $8\hat{P}_n$ y para $r = 2$ da $4\hat{P}_n$.

d) Aproximaciones de π y errores absolutos.

```
In [10]: data_3_pi = {}

for label, r in radios:
    rows = []
    for row in data_3[label]:
        n = row[0]
        p_n = row[2]
        pi_n = 16*p_n/r**2
        error = round(abs(pi_n - pi), 8)
        rows.append([n, p_n, pi_n, error])
    data_3_pi[label] = rows
```

```
In [11]: for label, r in radios:
          print(f"r = {label}")
```

```

print(tabulate(
    data_3_pi[label],
    headers=["n", "P_n(C_r) (E)", "Pi_n (E)", "error abs"],
    tablefmt="github"
))
print()

```

```

r = sqrt(2)

```

n	P_n(C_r) (E)	Pi_n (E)	error abs
10 ²	0.38	3.04	0.101593
10 ³	0.381	3.048	0.0935926
10 ⁴	0.39	3.12	0.0215927
10 ⁵	0.3923	3.1384	0.00319265
10 ⁶	0.392409	3.13927	0.00232065

```

r = 2

```

n	P_n(C_r) (E)	Pi_n (E)	error abs
10 ²	0.85	3.4	0.258407
10 ³	0.771	3.084	0.0575927
10 ⁴	0.7812	3.1248	0.0167926
10 ⁵	0.78542	3.14168	8.735e-05
10 ⁶	0.785792	3.14317	0.00157535

e) Comparación de los tres radios.

Para $r = 1$ se reutilizan las aproximaciones ya calculadas en los puntos 1 y 2 (`data_2`).

```

In [12]: data_cmp = []

for k in range(len(data_2)):
    n = data_2[k][0]
    pi_r1 = data_2[k][2] # viene del punto 2
    pi_r2 = data_3_pi["sqrt(2)"][k][2]
    pi_r3 = data_3_pi["2"][k][2]
    data_cmp.append([n, pi_r1, pi_r2, pi_r3])

print(tabulate(
    data_cmp,
    headers=["n", "Pi_n (r=1)", "Pi_n (r=sqrt(2))", "Pi_n (r=2)"],
    tablefmt="github"
))

```

n	Pi_n (r=1)	Pi_n (r=sqrt(2))	Pi_n (r=2)
10^2	3.52	3.04	3.4
10^3	3.36	3.048	3.084
10^4	3.1216	3.12	3.1248
10^5	3.12656	3.1384	3.14168
10^6	3.15403	3.13927	3.14317

```
In [13]: errores = []

for row in data_cmp:
    errores.append([row[0]] + [round(abs(v - pi), 8) for v in row[1:]])

print(tabulate(
    errores,
    headers=["n", "error (r=1)", "error (r=sqrt(2))", "error (r=2)"],
    tablefmt="github"
))
```

n	error (r=1)	error (r=sqrt(2))	error (r=2)
10^2	0.378407	0.101593	0.258407
10^3	0.218407	0.0935926	0.0575927
10^4	0.0199927	0.0215927	0.0167926
10^5	0.0150326	0.00319265	8.735e-05
10^6	0.0124394	0.00232065	0.00157535

f) ¿Cuál radio aproxima mejor?

El radio $r = 2$. La razón es la varianza del estimador. Como $\hat{P}_n(C_r)$ es una proporción binomial con $p = P(C_r) = \pi r^2/16$,

$$\text{Var}(\hat{\pi}_n) = \left(\frac{16}{r^2}\right)^2 \frac{p(1-p)}{n} = \frac{1}{n} \cdot \frac{16\pi}{r^2} \left(1 - \frac{\pi r^2}{16}\right).$$

Evaluando:

r	$p = P(C_r)$	$n \cdot \text{Var}(\hat{\pi}_n)$
1	$\pi/16 \approx 0.1963$	$16\pi - \pi^2 \approx 40.39$
$\sqrt{2}$	$\pi/8 \approx 0.3927$	$8\pi - \pi^2 \approx 15.26$
2	$\pi/4 \approx 0.7854$	$4\pi - \pi^2 \approx 2.70$

Es decir, la desviación estándar de $\hat{\pi}_n$ con $r = 2$ es cerca de $\sqrt{40.39/2.70} \approx 3.9$ veces menor que con $r = 1$, para el mismo n .

La intuición: el factor de amplificación $16/r^2$ multiplica el error de estimación de la proporción, y ese factor es máximo cuando el círculo es pequeño. Con $r = 1$ sólo cerca del 20% de los puntos cae dentro, así que la información útil por punto es baja y el error se multiplica por 16; con $r = 2$ el círculo está inscrito en el cuadrado, casi el 79% de los puntos cae dentro y el error sólo se multiplica por 4. Por eso el método clásico de Monte Carlo para π usa justamente el círculo inscrito.

(En una corrida particular puede pasar que un r menor dé un error más pequeño por azar; lo que se afirma aquí es sobre el error *esperado*, y se confirma al mirar la tendencia sobre todos los n .)